

Лабораторная работа №1 — финальная версия

У КАЖДОГО шага есть объяснение. Без исключений.

Формат каждого шага одинаковый:

- **Что печатаем** — точный текст
 - **Что это** — определение элемента языка
 - **Зачем** — зачем он тут нужен и что будет, если его не написать
-

ЧАСТЬ А. Создание проекта

Шаг 1

Что печатаем: `Win + R` , затем `cmd` , затем Enter. **Что это:** открытие командной строки Windows. **Зачем:** лабораторная требует создать проект именно через CLI (`dotnet`), а не через Visual Studio. Всё взаимодействие — через `cmd`.

Шаг 2

Что печатаем: `cd C:\` → Enter. **Что это:** команда `cd` (*change directory*) — «сменить папку». `C:\` — корень диска C. **Зачем:** по умолчанию `cmd` открывается в папке пользователя. Мы хотим предсказуемое место для всех работ.

Шаг 3

Что печатаем: `mkdir Labs` → Enter. **Что это:** `mkdir` (*make directory*) — создать папку. **Зачем:** получим одну папку `Labs` для всех лабораторных семестра.

Шаг 4

Что печатаем: `cd Labs` → Enter. **Что это:** переход внутрь папки `Labs` . **Зачем:** все следующие команды будут выполняться внутри `C:\Labs\` .

Шаг 5

Что печатаем: `dotnet new console --use-program-main -n ShopApp` → Enter. **Что это:** команда создания проекта. Разбор:

- `dotnet` — утилита .NET SDK;
- `new` — создать новый проект;
- `console` — по шаблону «консольное приложение»;
- `--use-program-main` — **важный флаг**, включает классическую точку входа `static void Main`. Без него .NET 8 создаст файл без явного `Main`;
- `-n ShopApp` — имя проекта. **Зачем:** нам нужна именно классическая точка входа по требованию лабораторной.

Шаг 6

Что печатаем: `cd ShopApp` → Enter. **Что это:** переход в папку созданного проекта. **Зачем:** команда `dotnet run` работает относительно текущей папки.

Шаг 7

Что делаем: в проводнике открываем `C:\Labs\ShopApp`, включаем показ расширений файлов.

Что это: настройка отображения в проводнике. **Зачем:** без этого файл `Product.cs` будет показан как `Product`, и при переименовании легко получить `Product.cs.txt`.

ЧАСТЬ В. Класс `Product`

Шаг 8

Что делаем: создаём пустой текстовый файл, переименовываем в `Product.cs`, открываем в блокноте. **Что это:** пустой файл для нового класса. **Зачем:** в C# принято «один класс — один файл», и имя файла = имя класса.

Шаг 9

Что печатаем:

```
namespace ShopApp;
```

Затем Enter. **Что это:**

- `namespace` — ключевое слово «объявляю пространство имён»;
- `ShopApp` — имя пространства (совпадает с именем проекта);

- `;` — это **file-scoped namespace**, современный синтаксис C# 10+. **Зачем:** пространство имён объединяет классы в группу. Класс `Program` уже в `ShopApp`, значит наш `Product` тоже попадёт туда — и в `Program.cs` не понадобится `using`. Без этого программа класс не найдёт.

Шаг 10

Что печатаем: ничего, просто Enter — пустая строка. **Что это:** пустая строка-разделитель.

Зачем: визуально отделяет «шапку» файла от тела класса. Компилятор её игнорирует, нужна человеку для читаемости.

Шаг 11

Что печатаем:

```
public class Product
```

Enter в конце, **без** `;` и **без** `{`. **Что это:**

- `public` — модификатор доступа «доступен отовсюду»;
- `class` — ключевое слово «объявляю класс»;
- `Product` — имя класса (PascalCase, с большой буквы). **Зачем:** `public` нужен, чтобы `Program.cs` мог создавать объекты этого класса. Имя класса задаёт тип — позже напишем `Product product = new Product();`. Без `;`, потому что следом идёт блок с `{ }`, а не инструкция.

Шаг 12

Что печатаем:

```
{
```

Затем Enter. **Что это:** открывающая фигурная скобка — начало тела класса. **Зачем:** всё между этой `{` и парной `}` в конце файла станет содержимым класса (поля, методы). Без закрывающей скобки компилятор скажет `} expected`.

Шаг 13

Что печатаем (4 пробела в начале — отступ):

```
    public string Name;
```

Затем Enter. **Что это:**

- 4 пробела — **отступ**, показывает вложенность (это внутри класса);
- `public` — поле доступно снаружи класса;
- `string` — **тип данных** «строка текста»;
- `Name` — имя поля;
- `;` — конец объявления. **Зачем:** поле хранит название товара. `public` нужен, чтобы в `Program.cs` написать `product.Name = "..."`. У поля **нет скобок и нет тела** — оно только хранит значение.

Шаг 14

Что печатаем:

```
public double Price;
```

Затем Enter. **Что это:**

- `double` — **тип данных** «вещественное число» (с дробной частью);
- `Price` — имя поля «цена». **Зачем:** цена может быть 99.99 — тип `int` не подошёл бы. `double` хранит дробные числа и легко принимает целые (30000 автоматически станет 30000.0).

Шаг 15

Что печатаем:

```
public int Quantity;
```

Затем Enter. **Что это:**

- `int` — **тип данных** «целое число»;
- `Quantity` — имя поля «количество». **Зачем:** количество штук — целое, дробным быть не может. Тип должен отражать смысл данных.

Шаг 16

Что печатаем:

```
public bool IsAvailable;
```

Затем Enter. **Что это:**

- `bool` — **тип данных** «логический», значения только `true` или `false` ;
 - `IsAvailable` — имя поля «доступен ли». Начинается с `Is` — соглашение для булевых.
- Зачем:** можно проверять `if (product.IsAvailable) ...`. Тип `bool` принимает строго два значения.

Мы получили **4 поля разных типов** — требование лабораторной.

Шаг 17

Что печатаем: ничего, просто Enter. **Что это:** пустая строка. **Зачем:** визуально отделяет блок полей от блока методов. Соглашение C#: сначала поля, потом методы.

Шаг 18

Что печатаем:

```
public double GetTotalPrice()
```

Enter, **без** `;` . **Что это:**

- `public` — метод доступен снаружи;
 - `double` — **тип возвращаемого значения** (метод вернёт дробное число);
 - `GetTotalPrice` — имя метода (PascalCase);
 - `()` — **круглые скобки обязательны для метода**. Пустые — значит, параметров нет.
- Зачем:** этот метод будет считать итоговую стоимость товара. Без `;` , потому что после сигнатуры идёт тело `{ }` .

Шаг 19

Что печатаем:

```
{
```

Затем Enter. **Что это:** открывающая фигурная скобка — начало **тела метода**. **Зачем:** код внутри этой `{` и парной `}` выполнится при вызове `GetTotalPrice()` .

Шаг 20

Что печатаем (8 пробелов отступ — три уровня вложенности):

```
return Price * Quantity;
```

Затем Enter. **Что это:**

- `return` — ключевое слово «вернуть значение»;
- `Price * Quantity` — выражение: `*` — умножение, берём поля текущего объекта;
- `;` — конец инструкции. **Зачем:** метод, объявленный как возвращающий `double`, **обязан** вернуть значение типа `double`. Если товар 30000 и 2 штуки, вернёт 60000. Это математика из пункта 6.

Шаг 21

Что печатаем:

```
}
```

Затем Enter. **Что это:** закрывающая фигурная скобка. **Зачем:** закрывает тело метода `GetTotalPrice`. Отступ 4 пробела — на уровне сигнатуры.

Шаг 22

Что печатаем: ничего, Enter. **Что это:** пустая строка. **Зачем:** визуально отделяет методы друг от друга.

Шаг 23

Что печатаем:

```
public void PrintInfo()
```

Затем Enter. **Что это:**

- `public` — доступен снаружи;
- `void` — «пустой», метод ничего не возвращает;
- `PrintInfo` — имя метода («печатать информацию»);
- `()` — параметров нет. **Зачем:** метод просто делает работу — печатает. `void` означает, что у него нет `return` со значением. Без `;`.

Шаг 24

Что печатаем:

```
{
```

Затем Enter. **Что это:** открывает тело метода `PrintInfo`. **Зачем:** внутри будут инструкции вывода.

Шаг 25

Что печатаем (это начало длинной строки, **Enter не жми** до шага 37):

```
Console.WriteLine(
```

Что это:

- `Console` — **стандартный класс** из .NET (пространство `System`), отвечает за работу с консолью;
- `.` — оператор обращения к члену класса;
- `WriteLine` — метод `Console`, печатает строку и переводит курсор на новую строку;
- `(` — открывает список аргументов. Мы передадим один аргумент — строку. **Зачем:** это единственный способ вывести что-то на экран. Пункт 3 лабораторной («работа со стандартными классами») реализуется именно здесь.

Шаг 26

Что печатаем (продолжаем ту же строку):

```
$"
```

Что это:

- `$` перед кавычкой — **включает интерполяцию**. Это фича C# 6+;
- `"` — открывающая кавычка строкового литерала. **Зачем:** без `$` внутри строки фигурные скобки `{}` воспринимались бы как обычный текст, и на экран вывелось бы буквально `{Name}`. С `$` — компилятор поймёт: это место подстановки.

Шаг 27

Что печатаем (продолжаем строку):

```
Товар:
```

(после двоеточия — пробел). **Что это:** обычный текст внутри интерполированной строки.

Зачем: формируем подпись «Товар: », после которой появится название.

Шаг 28

Что печатаем (продолжаем строку):

```
{Name}
```

Что это:

- { и } — **границы подстановки** в интерполированной строке;
- Name — **имя поля**, значение которого подставится. **Зачем:** в момент выполнения на место {Name} подставится название товара, например Смартфон . Кавычки внутри {} не нужны — это не строка, а обращение к полю.

Шаг 29

Что печатаем (продолжаем строку):

```
, цена:
```

Что это: обычный текст-разделитель. **Зачем:** отделяет название от следующего значения подписью «цена: ».

Шаг 30

Что печатаем (продолжаем строку):

```
{Price}
```

Что это: подстановка значения поля Price . **Зачем:** сюда подставится цена, например 30000 .

Шаг 31

Что печатаем (продолжаем строку):

```
руб. , кол-во:
```

Что это: текст: единица измерения + разделитель + подпись. **Зачем:** формируем читаемый вывод.

Шаг 32

Что печатаем (продолжаем строку):

```
{Quantity}
```

Что это: подстановка значения поля `Quantity`. **Зачем:** сюда подставится количество, например 2.

Шаг 33

Что печатаем (продолжаем строку):

```
шт., ИТОГО:
```

Что это: текст-разделитель. **Зачем:** подпись перед вызовом метода.

Шаг 34

Что печатаем (продолжаем строку):

```
{GetTotalPrice()}
```

Что это:

- `{` и `}` — границы подстановки;
- `GetTotalPrice` — имя метода того же класса;
- `()` — **обязательны**, потому что мы вызываем метод, а не читаем поле. **Зачем:** внутри интерполяции можно не только имя поля, но и **вызов метода**. `C#` вызовет метод, получит `double`, приведёт к строке. Пункт 5 лабораторной («публичные методы») демонстрируется прямо здесь.

Шаг 35

Что печатаем (продолжаем строку):

```
руб.
```

Что это: текст после подстановки — единица измерения. **Зачем:** закрываем смысловую часть вывода.

Шаг 36

Что печатаем (продолжаем строку):

```
"
```

Что это: **закрывающая двойная кавычка** — конец строкового литерала. **Зачем:** без неё компилятор будет считать, что строка продолжается, и весь остальной файл «уйдёт в строку». Ошибка вида `Newline in constant`.

Шаг 37

Что печатаем (продолжаем строку):

```
);
```

Затем **Enter**. **Что это:**

- `)` — закрывает список аргументов `Console.WriteLine` ;
- `;` — завершает инструкцию. **Зачем:** теперь инструкция завершена, можно переходить на новую строку. Проверь: у тебя должна получиться ровно одна строка целиком — если она разорвана переносом, C# не поймёт.

Шаг 38

Что печатаем:

```
}
```

Затем Enter. **Что это:** закрывающая скобка тела метода `PrintInfo`. **Зачем:** отмечает конец метода. Отступ 4 пробела.

Шаг 39

Что печатаем:

```
}
```

Затем Enter. **Что это:** закрывающая скобка тела класса `Product`. **Зачем:** отмечает конец класса. Отступ 0 пробелов — на уровне `public class Product`.

Сохрани `Product.cs` (`Ctrl+S`). Не запускаем — впереди ещё три класса.

ЧАСТЬ С. Класс Customer

Шаг 40

Что делаем: создаём пустой `Customer.cs`, открываем. **Что это:** новый файл под новый класс.

Зачем: имя файла = имя класса.

Шаг 41

Что печатаем:

```
namespace ShopApp;
```

Enter, потом Enter (пустая строка). **Что это:** объявление того же пространства имён. **Зачем:** критично, что имя совпадает с `ShopApp`. Если написать другое — `Program.cs` не найдёт класс без `using`. Пустая строка — разделитель.

Шаг 42

Что печатаем:

```
public class Customer
```

Enter, без `;`. **Что это:** объявление публичного класса `Customer` («Покупатель»). **Зачем:** `public` нужен для доступа из `Program.cs`. Без `;` — потому что дальше блок.

Шаг 43

Что печатаем:

```
{
```

Enter. **Что это:** начало тела класса. **Зачем:** открывает содержимое класса.

Шаг 44

Что печатаем:

```
public string FullName;
```

Enter. **Что это:** поле «полное имя» типа `string`, публичное. **Зачем:** здесь будет ФИО покупателя. Отступ 4 пробела.

Шаг 45

Что печатаем:

```
public double Balance;
```

Enter. **Что это:** поле «баланс» типа `double`, публичное. **Зачем:** деньги могут быть с копейками. `public`, чтобы из `Program.cs` задать начальный баланс и потом читать остаток.

Шаг 46

Что печатаем:

```
public int Age;
```

Enter. **Что это:** поле «возраст», тип `int`. **Зачем:** возраст — целые годы.

Шаг 47

Что печатаем: ничего, Enter. **Что это:** пустая строка-разделитель. **Зачем:** отделяет поля от методов.

Шаг 48

Что печатаем:

```
public bool CanBuy(double price)
```

Enter, без `;`. **Что это:**

- `public` — доступен снаружи;
- `bool` — **метод вернёт true/false**;
- `CanBuy` — имя («может купить»);
- `(` открывает список параметров;
- `double price` — **объявление параметра:** тип `double`, имя `price`. Внутри метода это переменная, которой при вызове передадут значение;
- `)` закрывает список. **Зачем:** метод отвечает на вопрос «хватает ли денег». Параметр нужен, чтобы один и тот же метод мог работать с любой ценой.

Шаг 49

Что печатаем:

```
{
```

Enter. **Что это:** начало тела метода `CanBuy`. **Зачем:** внутри будет одна инструкция `return`.

Шаг 50

Что печатаем:

```
return Balance >= price;
```

Enter. **Что это:**

- `return` — вернуть значение;
- `Balance >= price` — **условие:** `>=` — оператор сравнения «больше или равно». Не путать с `=` (присваивание!);
- результат: `true` или `false` — совпадает с типом возврата;
- `;`. **Зачем:** метод получит цену, сравнит с балансом и вернёт ответ. Отступ 8 пробелов.

Шаг 51

Что печатаем:

```
}
```

Enter. **Что это:** закрывает метод `CanBuy`.

Шаг 52

Что печатаем: ничего, Enter. **Что это:** пустая строка-разделитель.

Шаг 53

Что печатаем:

```
public void Pay(double amount)
```

Enter, без `;`. **Что это:**

- `public` — доступен снаружи;
- `void` — **ничего не возвращает**;
- `Pay` — имя («оплатить»);
- `double amount` — параметр «сумма к списанию». **Зачем:** метод изменяет состояние объекта (уменьшает баланс), но ему нечего возвращать.

Шаг 54

Что печатаем:

```
{
```

Enter. **Что это:** начало тела метода `Pay` .

Шаг 55

Что печатаем:

```
    Balance -= amount;
```

Enter. **Что это:**

- `-=` — **составной оператор присваивания**. Сокращение от `Balance = Balance - amount;` ;
- `;` . **Зачем:** так принято в C#. **Не путать с** `=-` : если написать `Balance -= amount;` , компилятор поймёт это как `Balance = -amount;` — присвоить отрицательное. Классическая ловушка.

Шаг 56

Что печатаем:

```
}
```

Enter. **Что это:** закрывает метод `Pay` .

Шаг 57

Что печатаем: ничего, Enter. **Что это:** пустая строка.

Шаг 58

Что печатаем:

```
public void PrintInfo()
```

Enter. **Что это:** метод вывода, `void` . **Зачем:** печатает данные покупателя.

Шаг 59

Что печатаем:

```
{
```

Enter.

Шаг 60

Что печатаем (Enter не жми — длинная строка):

```
Console.WriteLine(
```

Что это: начинаем вывод через стандартный класс `Console` . **Зачем:** печатаем информацию в консоль.

Шаг 61

Что печатаем (продолжаем):

```
$"Покупатель:"
```

Что это: `$` включает интерполяцию, `"` открывает строку, дальше текст. **Зачем:** без `$` фигурные скобки ниже были бы обычным текстом.

Шаг 62

Что печатаем (продолжаем):

```
{FullName}
```

Что это: подстановка значения поля `FullName` . **Зачем:** вместо `{FullName}` в выводе появится имя.

Шаг 63

Что печатаем (продолжаем):

```
, возраст:
```

Что это: текст-разделитель. **Зачем:** подпись перед следующим значением.

Шаг 64

Что печатаем (продолжаем):

```
{Age}
```

Что это: подстановка поля `Age` . **Зачем:** выведет возраст.

Шаг 65

Что печатаем (продолжаем):

```
, баланс:
```

Что это: текст-разделитель.

Шаг 66

Что печатаем (продолжаем):

```
{Balance}
```

Что это: подстановка поля `Balance` .

Шаг 67

Что печатаем (продолжаем):

```
руб. "
```

Затем Enter. **Что это:** текст + закрывающая кавычка. Кавычка закрывает строковый литерал.

Зачем: без неё компилятор будет считать, что строка продолжается. Enter — конец строки.

Шаг 68

Что печатаем:

```
);
```

Enter. **Что это:** `)` закрывает список аргументов `WriteLine`, `;` завершает инструкцию.

Шаг 69

Что печатаем:

```
}
```

Enter. **Что это:** закрывает метод `PrintInfo`.

Шаг 70

Что печатаем:

```
}
```

Enter. **Что это:** закрывает класс `Customer`. **Зачем:** конец содержимого класса.

Сохрани `Customer.cs`.

ЧАСТЬ D. Класс `Discount`

Шаг 71

Что делаем: создаём `Discount.cs`, открываем. **Что это:** файл под третий класс. **Зачем:** имя = имя класса.

Шаг 72

Что печатаем:

```
namespace ShopApp;
```

Enter, Enter. **Что это:** то же пространство имён. **Зачем:** обязательно `ShopApp`, иначе `Program` не увидит класс.

Шаг 73

Что печатаем:

```
public class Discount
```

Enter, без ; . **Что это:** объявление публичного класса Discount («Скидка»).

Шаг 74

Что печатаем:

```
{
```

Enter. **Что это:** начало тела класса.

Шаг 75

Что печатаем:

```
public string Name;
```

Enter. **Что это:** поле «название скидки» типа string . **Зачем:** чтобы различать скидки, например «Осенняя распродажа».

Шаг 76

Что печатаем:

```
public double Percent;
```

Enter. **Что это:** поле «процент скидки», тип double . **Зачем:** double позволяет и 10, и 7.5.

Шаг 77

Что печатаем: ничего, Enter. **Что это:** пустая строка-разделитель.

Шаг 78

Что печатаем:

```
public double Apply(double price)
```

Enter, без ; . **Что это:**

- `double` — метод вернёт число;
- `Apply` — имя («применить»);
- `double price` — параметр «исходная цена». **Зачем:** метод посчитает цену со скидкой по переданной цене.

Шаг 79

Что печатаем:

```
{
```

Enter. **Что это:** начало тела метода `Apply` .

Шаг 80

Что печатаем:

```
return price - price * Percent / 100.0;
```

Enter. **Что это — по частям:**

- `price * Percent` — «сколько процентов» в числовом виде;
- `/ 100.0` — **делим на 100.0, а не на 100!** Если `Percent = 10` , а `price = 30000` , то `price * Percent = 300000` , делим на `100.0` → `3000.0` — это скидка в рублях. Если бы написали `100` , в некоторых случаях получили бы целочисленное деление и потеряли дробь;
- `price - ...` — вычитаем скидку из исходной цены, получаем итог;
- `return` — возвращаем результат;
- `;` . **Зачем:** ключевая математика лабораторной.

Шаг 81

Что печатаем:

```
}
```

Enter. **Что это:** закрывает метод `Apply` .

Шаг 82

Что печатаем: ничего, Enter.

Шаг 83

Что печатаем:

```
public void PrintInfo()
```

Enter. **Что это:** метод вывода, void .

Шаг 84

Что печатаем:

```
{
```

Enter.

Шаг 85

Что печатаем (Enter не жми):

```
Console.WriteLine(
```

Что это: начинаем вывод.

Шаг 86

Что печатаем (продолжаем):

```
$"Скидка:
```

Что это: \$ включает интерполяцию, " открывает строку, текст.

Шаг 87

Что печатаем (продолжаем):

```
{Name}
```

Что это: подстановка поля Name .

Шаг 88

Что печатаем (продолжаем):

```
, размер:
```

Что это: текст-разделитель.

Шаг 89

Что печатаем (продолжаем):

```
{Percent}
```

Что это: подстановка поля `Percent` .

Шаг 90

Что печатаем (продолжаем):

```
%"
```

Затем Enter. **Что это:** символ `%` (просто текст, не вычисление) и закрывающая кавычка. **Зачем:** кавычка завершает литерал.

Шаг 91

Что печатаем:

```
);
```

Enter. **Что это:** `)` закрывает `WriteLine` , `;` завершает.

Шаг 92

Что печатаем:

```
}
```

Enter. **Что это:** закрывает метод.

Шаг 93

Что печатаем:

```
}
```

Enter. **Что это:** закрывает класс `Discount` .

Сохрани `Discount.cs` .

ЧАСТЬ Е. Класс `Store`

Шаг 94

Что делаем: создаём `Store.cs` , открываем. **Что это:** файл под четвёртый класс.

Шаг 95

Что печатаем:

```
namespace ShopApp;
```

Enter, Enter. **Что это:** то же пространство имён. **Зачем:** чтобы `Program` видел класс.

Шаг 96

Что печатаем:

```
public class Store
```

Enter, без `;` . **Что это:** публичный класс `Store` («Магазин»).

Шаг 97

Что печатаем:

```
{
```

Enter. **Что это:** начало тела класса.

Шаг 98

Что печатаем:

```
public string StoreName;
```

Enter. **Что это:** поле «название магазина», тип `string`. **Зачем:** назвал `StoreName`, а не `Name`, чтобы не путалось с `Name` в других классах.

Шаг 99

Что печатаем:

```
public string City;
```

Enter. **Что это:** поле «город», тип `string`.

Шаг 100

Что печатаем:

```
public double Rating;
```

Enter. **Что это:** поле «рейтинг», тип `double`. **Зачем:** рейтинг бывает дробный, например 4.7.

Шаг 101

Что печатаем: ничего, Enter.

Шаг 102

Что печатаем:

```
public void PrintInfo()
```

Enter. **Что это:** метод вывода, `void`.

Шаг 103

Что печатаем:

```
{
```

Enter.

Шаг 104

Что печатаем (Enter не жми):

```
Console.WriteLine(
```

Шаг 105

Что печатаем (продолжаем):

```
$"Магазин:
```

Что это: \$ включает интерполяцию, текст.

Шаг 106

Что печатаем (продолжаем):

```
{StoreName}
```

Что это: подстановка поля `StoreName` .

Шаг 107

Что печатаем (продолжаем):

```
, город:
```

Шаг 108

Что печатаем (продолжаем):

```
{City}
```

Что это: подстановка поля `City` .

Шаг 109

Что печатаем (продолжаем):

```
, рейтинг:
```

Шаг 110

Что печатаем (продолжаем):

```
{Rating}
```

Что это: подстановка поля `Rating` .

Шаг 111

Что печатаем (продолжаем):

```
"
```

Затем Enter. **Что это:** закрывающая кавычка — конец строкового литерала.

Шаг 112

Что печатаем:

```
);
```

Enter.

Шаг 113

Что печатаем:

```
}
```

Enter. **Что это:** закрывает метод.

Шаг 114

Что печатаем:

```
}
```

Enter. **Что это:** закрывает класс `Store` .

Сохрани `Store.cs` . Все 4 класса готовы.

ЧАСТЬ F. Собираем всё в Program.cs

Шаг 115

Что делаем: открываем Program.cs , находим строку Console.WriteLine("Hello, World!"); , удаляем её целиком. **Что это:** убираем демо-строку из шаблона. **Зачем:** метод Main станет пустым. Все дальнейшие шаги — **внутри Main , между его { и } .**

Шаг 116

Что печатаем:

```
Store store = new Store();
```

Enter. **Что это по частям:**

- Store — тип (имя класса);
- store — имя переменной;
- = — присваивание;
- new — **создать объект**. Выделяет память, инициализирует поля значениями по умолчанию (0 , null , false);
- Store() — вызов конструктора (пока без параметров, генерируется автоматически);
- ; . **Зачем:** это **первый из четырёх объектов** лабораторной. Без new компилятор бы сказал «нельзя использовать переменную без присваивания».

Шаг 117

Что печатаем:

```
store.StoreName = "Электроника+";
```

Enter. **Что это:**

- store — наша переменная;
- . — **оператор доступа к члену;**
- StoreName — **публичное поле**. Если бы было private , компилятор запретил бы;
- = — присваивание;
- "Электроника+" — строковый литерал в двойных кавычках. **Не** в одинарных — те для одного символа;
- ; . **Зачем:** задаём имя магазина.

Шаг 118

Что печатаем:

```
store.City = "Москва";
```

Enter. **Что это:** присваивание значения полю `City`. **Зачем:** задаём город.

Шаг 119

Что печатаем:

```
store.Rating = 4.7;
```

Enter. **Что это:** присваиваем число с точкой. Точка — **десятичный разделитель** в C#, не запятая. **Зачем:** задаём рейтинг.

Шаг 120

Что печатаем:

```
store.PrintInfo();
```

Enter. **Что это:**

- `.PrintInfo` — обращение к методу;
- `()` — **вызов** метода. Без скобок это ссылка на метод, а не вызов;
- `;`. **Зачем:** метод выведет информацию о магазине.

Шаг 121

Что печатаем:

```
Console.WriteLine();
```

Enter. **Что это:** `WriteLine` **без аргументов** печатает пустую строку. **Зачем:** визуально отделяем блок вывода магазина от блока товара.

Шаг 122

Что печатаем:

```
Product product = new Product();
```

Enter. **Что это:** создание **второго объекта**. **Зачем:** все поля пока дефолтные.

Шаг 123

Что печатаем:

```
product.Name = "Смартфон";
```

Enter. **Что это:** присваивание значения полю `Name` .

Шаг 124

Что печатаем:

```
product.Price = 30000;
```

Enter. **Что это:** присваивание `int` -литерала в `double` -поле. **Неявное преобразование** `int` → `double` безопасно. **Зачем:** задаём цену.

Шаг 125

Что печатаем:

```
product.Quantity = 2;
```

Enter. **Что это:** присваивание количества.

Шаг 126

Что печатаем:

```
product.IsAvailable = true;
```

Enter. **Что это:** `true` — **литерал булева типа**. С маленькой буквы, **без кавычек**. **Зачем:** `"true"` — это строка, а не логическое значение. Компилятор бы выдал ошибку.

Шаг 127

Что печатаем:

```
product.PrintInfo();
```

Enter. **Что это:** вызов метода печати у товара.

Шаг 128

Что печатаем:

```
Discount discount = new Discount();
```

Enter. **Что это:** создание **третьего объекта**.

Шаг 129

Что печатаем:

```
discount.Name = "Осенняя распродажа";
```

Enter.

Шаг 130

Что печатаем:

```
discount.Percent = 10;
```

Enter.

Шаг 131

Что печатаем:

```
discount.PrintInfo();
```

Enter.

Шаг 132

Что печатаем:

```
double newPrice = discount.Apply(product.Price);
```

Enter. **Что это по частям:**

- `double newPrice` — объявление новой переменной типа `double` ;
- `=` — присваивание;
- `discount` — объект скидки;
- `.Apply` — вызов метода;
- `(product.Price)` — **аргумент**: внутрь идёт цена товара;
- `;` . **Зачем**: метод `Apply` вернёт 27000, значение сохранится в `newPrice` .

Шаг 133

Что печатаем (Enter не жми):

```
Console.WriteLine(
```

Шаг 134

Что печатаем (продолжаем):

```
$"Цена со скидкой:
```

Шаг 135

Что печатаем (продолжаем):

```
{newPrice}
```

Enter. **Что это**: подстановка локальной переменной, а не поля. Работает так же.

Шаг 136

Что печатаем (продолжаем):

```
руб. "
```

Enter.

Шаг 137

Что печатаем:

```
);
```

Enter.

Шаг 138

Что печатаем:

```
Console.WriteLine();
```

Enter. **Что это:** пустая строка-разделитель.

Шаг 139

Что печатаем:

```
Customer customer = new Customer();
```

Enter. **Что это:** создание **четвёртого объекта**.

Шаг 140

Что печатаем:

```
customer.FullName = "Иванов Иван";
```

Enter.

Шаг 141

Что печатаем:

```
customer.Balance = 50000;
```

Enter.

Шаг 142

Что печатаем:

```
customer.Age = 25;
```

Enter.

Шаг 143

Что печатаем:

```
customer.PrintInfo();
```

Enter.

Шаг 144

Что печатаем:

```
double total = newPrice * product.Quantity;
```

Enter. **Что это:** `newPrice` (27000) × `quantity` (2). Результат — 54000. **Зачем:** итоговая сумма к оплате.

Шаг 145

Что печатаем (Enter не жми):

```
Console.WriteLine(
```

Шаг 146

Что печатаем (продолжаем):

```
 $"Итого к оплате:
```

Шаг 147

Что печатаем (продолжаем):

```
 {total}
```

Enter.

Шаг 148

Что печатаем (продолжаем):

```
руб."
```

Enter.

Шаг 149

Что печатаем:

```
);
```

Enter.

Шаг 150

Что печатаем:

```
if (customer.CanBuy(total))
```

Enter. **Что это по частям:**

- `if` — **условный оператор**;
- `(` — начало условия;
- `customer.CanBuy(total)` — вызов метода, возвращающего `bool`. **Результат можно ставить прямо в условие**;
- `total` — аргумент;
- `)` — конец условия;
- **без** `;`, потому что после `if (...)` идёт блок `{ }`. **Зачем:** если баланс ≥ 54000 , пойдёт по ветке «успех», иначе — по `else`.

Шаг 151

Что печатаем:

```
{
```

Enter. **Что это:** открывающая скобка блока `if`. **Зачем:** открывает код, который выполнится, если условие истинно.

Шаг 152

Что печатаем (12 пробелов отступ — три уровня вложенности: Main → if → тело):

```
customer.Pay(total);
```

Enter. **Что это:** вызов метода `Pay` — спишет сумму с баланса. **Зачем:** внутри выполнится `Balance -= amount`.

Шаг 153

Что печатаем (Enter не жми):

```
Console.WriteLine(
```

Шаг 154

Что печатаем (продолжаем):

```
$"Покупка успешна! Остаток:
```

Шаг 155

Что печатаем (продолжаем):

```
{customer.Balance}
```

Enter. **Что это:** читаем **публичное** поле `Balance` **чужого объекта** после списания. **Зачем:** если бы поле было `private`, компилятор запретил бы чтение.

Шаг 156

Что печатаем (продолжаем):

```
руб."
```

Enter.

Шаг 157

Что печатаем:

```
);
```

Enter.

Шаг 158

Что печатаем:

```
}
```

Enter. **Что это:** закрывает блок `if` .

Шаг 159

Что печатаем:

```
else
```

Enter. **Что это:** ключевое слово «иначе». **Зачем:** сюда попадём, если `CanBuy` вернул `false` .

Между `}` и `else` **не ставим** `;` .

Шаг 160

Что печатаем:

```
{
```

Enter. **Что это:** открывающая скобка блока `else` .

Шаг 161

Что печатаем:

```
Console.WriteLine("Недостаточно средств.");
```

Enter. **Что это:** вывод строки. **Здесь нет** `$` , потому что подстановок нет. **Зачем:** сообщаем пользователю, что денег не хватает.

Шаг 162

Что печатаем:

```
}
```

Enter. **Что это:** закрывает блок `else` . Теперь `Main` полностью собран.

Сохрани `Program.cs` .

ЧАСТЬ G. Запуск

Шаг 163

Что делаем: в `cmd` (при необходимости открой заново и выполни `cd C:\Labs\ShopApp`) печатаем:

```
dotnet run
```

Enter. **Что это:** команда компилирует проект (превращает `.cs` в `.dll`) и запускает его. **Зачем:** проверяем результат. При ошибках компилятор покажет файл и номер строки.

Ожидаемый вывод:

```
Магазин: Электроника+, город: Москва, рейтинг: 4.7
```

```
Товар: Смартфон, цена: 30000 руб., кол-во: 2 шт., итого: 60000 руб.
```

```
Скидка: Осенняя распродажа, размер: 10%
```

```
Цена со скидкой: 27000 руб.
```

```
Покупатель: Иванов Иван, возраст: 25, баланс: 50000 руб.
```

```
Итого к оплате: 54000 руб.
```

```
Недостаточно средств.
```

Почему «Недостаточно средств»: $50000 < 54000$, `CanBuy` вернул `false` , сработал `else` .

Шаг 164

Что делаем: в `Program.cs` находим `customer.Balance = 50000;` , меняем на `60000` . Сохраняем, запускаем. **Зачем:** проверяем ветку `if` . Теперь `CanBuy` вернёт `true` , `Pay(54000)` уменьшит баланс до `6000` , выведется строка «Покупка успешна!».

ЧАСТЬ Н. Самостоятельная работа

Задание 1. Новое поле в `Product`

- В `Product.cs` найди `public bool IsAvailable;`. С новой строки ниже напечатай:
 - `public string Manufacturer;` — **что это:** пятое поле, «производитель», тип `string`.
Зачем: демонстрирует, что класс можно расширять новыми полями.
- В `PrintInfo` найди `$"Товар: "`. Поставь курсор сразу после `$"` и напечатай на той же строке:
 - `Производитель: "` — текст-подпись;
 - `{Manufacturer}` — подстановка нового поля;
 - `,` — разделитель. **Зачем:** добавляем ещё одну подстановку в шаблон.
- В `Program.cs` найди `product.Name = "Смартфон";`. С новой строки ниже напечатай:
 - `product.Manufacturer = "Samsung";` — присваиваем значение новому полю. **Зачем:** без присваивания поле останется `null`.
- Сохрани, запусти.

Задание 2. Новый метод в `Customer`

- В `Customer.cs` перед закрывающей `}` класса напечатай по строкам:
 - `public int GetAgeInMonths()` — **что это:** `public` — доступен снаружи; `int` — вернёт целое; имя говорит за себя; `()` — без параметров. **Зачем:** считаем возраст в месяцах.
 - `{` — открывает тело метода.
 - `return Age * 12;` — **что это:** умножаем `Age` на 12 (месяцев в году), возвращаем результат. **Зачем:** демонстрируем ещё одну математическую операцию.
 - `}` — закрывает метод.
- В `Program.cs` найди `customer.PrintInfo();`. С новой строки ниже напечатай:
 - `Console.WriteLine(` — начинаем вывод;
 - `$"Возраст в месяцах: "` — включаем интерполяцию, пишем текст;
 - `{customer.GetAgeInMonths()}` — **что это:** в интерполяции вызываем метод **чужого объекта** через точку. **Зачем:** C# вызовет метод, получит `int`, приведёт к строке;
 - `");` — закрывающая кавычка, скобка `WriteLine`, `;`.
- Сохрани, запусти.

Задание 3. Новый метод в `Discount`

1. В `Discount.cs` перед закрывающей `}` класса напечатай по строкам:

- `public double ApplyToQuantity(double price, int quantity)` — **что это:** метод вернёт `double`, принимает два параметра (цену и количество). **Зачем:** инкапсулируем расчёт стоимости партии внутри класса скидки;
- `{` — тело;
- `return Apply(price) * quantity;` — **что это:** вызываем уже существующий `Apply`, получаем цену со скидкой, умножаем на количество. **Зачем:** переиспользуем код, не дублируем формулу;
- `}`.

2. В `Program.cs` найди `double total = newPrice * product.Quantity;`. Замени её на:

- `double total = discount.ApplyToQuantity(product.Price, product.Quantity);` — **что это:** теперь расчёт целиком в классе `Discount`. **Зачем:** правильный ООП-подход — `Program.cs` только вызывает метод.

3. Сохрани, запусти — результат прежний.

Чек-лист

- ☐ Проект создан через `dotnet new console --use-program-main`.
- ☐ `Main` — классическая точка входа.
- ☐ 4 класса: `Product`, `Customer`, `Discount`, `Store`.
- ☐ 4 объекта: `store`, `product`, `discount`, `customer`.
- ☐ Поля типов `string`, `double`, `int`, `bool`.
- ☐ Методы: `GetTotalPrice`, `CanBuy`, `Pay`, `Apply`, `PrintInfo`.
- ☐ Математика: `*`, `-`, `>=`, `/ 100.0`.
- ☐ Все подстановки — через `$"..."`.
- ☐ `dotnet run` без ошибок.